

SMART CAMPUS EVENT MANAGEMENT SYSTEM

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

CHALLA PREM SEKHAR (VTU25118)

10211CS224

Full Stack Application Development

Slot : E1-B2



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D INSTITUTE OF
SCIENCE AND TECHNOLOGY**

**(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

May, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled "Smart Campus Event Managaement System" by CHALLA PREM SEKHAR (VTU25118) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr.Hemamalini.S

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(CHALLA PREM SEKHAR)

Date: 06/05/2026

ABSTRACT

CampusEvents is a role-based campus event management system developed to simplify and digitize the process of organizing and participating in campus events. Traditional methods of event management often rely on manual processes, which are time-consuming, error-prone, and lack real-time visibility. This project addresses these challenges by providing a centralized web-based platform for both administrators and students.

The system is built using a full-stack architecture with a frontend developed using HTML, CSS, Bootstrap, and Thymeleaf, and a backend powered by Spring Boot and Spring Security. It implements role-based authentication, allowing administrators to create, update, and manage events, while students can browse, register, and track their event participation.

Key features include real-time capacity tracking to prevent overbooking, personalized dashboards for users, and an intuitive user interface for seamless interaction. The system uses a MySQL database to store user, event, and registration data securely.

Overall, the CampusEvents platform improves efficiency, enhances user experience, and provides a scalable solution for modern campus event management.

Keywords: Campus Event Management, Full-Stack Web Application, Role-Based Access Control, Spring Boot, Spring Security, Event Registration System, Real-Time Capacity Tracking, MySQL Database, Web Application Development, User Dashboard

LIST OF FIGURES

1.1	Architecture of CampusEvents System	7
4.1	Data Flow Diagram Level 0	16
4.2	Data Flow Diagram Level 1	17
5.1	Admin Authentication	23
5.2	Admin Dashboard	24
5.3	Student Event Browsing	25
5.4	Event Registration Page	26
5.5	Student Dashboard	27

LIST OF TABLES

2.1	Comparative Analysis of Event Management Systems	10
3.1	System Technology Stack for CampusEvents	13
7.1	Mapping of Project to Complex Engineering Problem (CEP)	36
7.2	Mapping of Project to Complex Engineering Activities (CEA) . . .	37
7.3	SDG Mapping for CampusEvents	38

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CSS	Cascading Style Sheets
DBMS	Database Management System
HTML	HyperText Markup Language
JPA	Java Persistence API
MVC	Model View Controller
REST	Representational State Transfer
SDG	Sustainable Development Goal
SQL	Structured Query Language

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	2
1.3 Scope of the Project	2
1.4 Functional Scope	3
1.4.1 Functional Scope	3
1.5 Technical and Architectural Scope	3
1.6 Limitations and Out of Scope	4
1.7 Frameworks and Technologies	5
1.7.1 Backend Framework: Spring Boot	5
1.7.2 Frontend Framework: Thymeleaf	6
1.7.3 Frontend Styling	6
1.7.4 Database and Persistence: MySQL	6
2 Survey of Similar Applications in Market	8
2.0.1 Large-Scale Public Event Platforms (Eventbrite, Meetup) . .	8

2.0.2	University Traditional Methods (Google Forms, Notice Boards)	9
2.0.3	Modern Web-Based Systems	9
2.0.4	Gap Analysis and CampusEvents Advantage	9
3	Framework Description	11
3.1	Frontend Implementation	11
3.1.1	Environment Setup	11
3.1.2	Frontend Features	11
3.2	Backend Implementation	12
3.2.1	Environment Setup	12
3.2.2	Backend Features	12
3.3	Database Implementation	12
3.3.1	Database Configuration	12
3.3.2	Database Schema Structure	13
3.4	System Technology Stack Overview	13
4	METHODOLOGIES	14
4.1	Project Architecture	14
4.2	Data Flow Diagram (DFD)	15
4.2.1	DFD Level 0: Context Diagram	15
4.2.2	DFD Level 1: Functional Decomposition	16
4.3	System Modules and Functional Architecture	17
4.3.1	Module 1: User Authentication and Security	18
4.3.2	Module 2: Event Management Module	18
4.3.3	Module 3: Event Browsing Module	18
4.3.4	Module 4: Registration Module	19

4.3.5	Module 5: Dashboard Module	19
4.3.6	Module 6: Capacity Tracking Module	19
4.3.7	Module 7: Database Management Module	20
4.4	Advantages and Limitations of the Project	20
4.4.1	Advantages	20
4.4.2	Limitations	20

5 RESULTS AND DISCUSSIONS 21

5.1	System Implementation Overview	21
5.2	System Interface Visualizations	22
5.3	Module Explanations	27
5.3.1	Landing Page Interface	28
5.3.2	User Authentication Module	28
5.3.3	Event Browsing Module	28
5.3.4	Event Details Module	28
5.3.5	Event Registration Module	28
5.3.6	Student Dashboard Module	28
5.3.7	Admin Dashboard Module	28
5.3.8	Event Management Module	28
5.3.9	Capacity Tracking Module	29
5.3.10	Database Management Module	29
5.3.11	Search and Filtering Module	29
5.3.12	Security Module	29
5.3.13	System Logging Module	29

6 CONCLUSION AND FUTURE ENHANCEMENTS 30

6.1	Conclusion	30
-----	----------------------	----

6.2	Future Enhancements	31
6.3	Source Code	32
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	35
7.1	Mapping of the Project to Complex Engineering Problem (CEP) . .	35
7.2	Mapping of the Project to Complex Engineering Activities (CEA) .	36
7.3	SDG Mapping (CEP Section)	37
	References	39

Chapter 1

INTRODUCTION

1.1 Introduction

In today's digital era, the efficient management of events has become an essential requirement for educational institutions. Colleges and universities regularly organize a wide range of events such as workshops, seminars, hackathons, technical fests, and cultural programs. These events play a crucial role in enhancing students' technical knowledge, interpersonal skills, and overall development. However, the management of such events is often carried out using traditional methods like manual registrations, spreadsheets, or fragmented digital tools such as Google Forms [3], which lead to inefficiencies, errors, and lack of coordination.

Manual event management systems are time-consuming and prone to human errors such as duplicate entries, incorrect data handling, and miscommunication between organizers and participants. Additionally, students often face difficulties in discovering relevant events due to the absence of a centralized platform. This results in missed opportunities and reduced participation. On the other hand, administrators struggle to monitor registrations, manage event capacities, and update event details in real time, unlike modern platforms such as Eventbrite and Meetup [1][2]. These challenges highlight the need for an automated, reliable, and user-friendly event management solution.

To address these issues, the CampusEvents: Role-Based Campus Event Management Platform has been developed as a full-stack web application. The system is built using modern technologies such as Spring Boot [4], Spring Security [5], Thymeleaf [6], and MySQL [8]. The primary aim of this system is to streamline

the process of event creation, management, and participation through a centralized digital platform. The application provides separate interfaces for administrators and students, ensuring a clear separation of responsibilities through role-based access control.

1.2 Aim of the Project

The aim of this project is to design and develop a centralized web-based campus event management system that simplifies the process of organizing and participating in events. It focuses on providing a user-friendly platform for both administrators and students through role-based access control using Spring Security [5]. The system enables administrators to create, update, and manage events efficiently while allowing students to easily browse and register for events. It ensures secure authentication using modern frameworks like Spring Security [5].

The project aims to eliminate manual errors and improve data accuracy through automated processes. It also provides real-time capacity tracking to prevent overbooking of events. Additionally, the system enhances communication and visibility between organizers and participants. The platform is designed to be responsive and accessible across different devices using technologies such as Bootstrap [10], HTML [11], and CSS [12]. Overall, the goal is to improve efficiency, transparency, and user experience in campus event management.

1.3 Scope of the Project

The scope of the CampusEvents project is to develop a centralized web-based platform that efficiently manages campus events from creation to participation. It includes functionalities for both administrators and students through role-based access control [5], where administrators can create, update, and manage events while monitoring registrations and capacity in real time, and students can browse, filter,

and register for events through an intuitive interface. The system ensures secure authentication [5] and maintains all data in a structured database for reliability and accuracy using MySQL [8]. It supports real-time updates to prevent overbooking and provides personalized dashboards for better user experience. The platform is designed to be responsive and accessible across multiple devices using technologies such as Bootstrap [10], HTML [11], and CSS [12], reducing manual effort and improving efficiency. The project can be further extended with features like email notifications, payment gateway integration, cloud deployment, analytics, and mobile app support, making it a scalable solution adaptable to other organizations beyond educational institutions.

1.4 Functional Scope

1.4.1 Functional Scope

The primary functional features of the system include:

- **User Authentication:** Secure login using Spring Security with Role-Based Access Control (Admin/Student).
- **Event Management:** Administrators can create, update, delete, and monitor campus events efficiently.
- **Registration System:** Students can browse available events, register easily, and track their participation.
- **Dashboard:** Personalized dashboards for both Admin and Student to view event data and statistics.

1.5 Technical and Architectural Scope

- **Full Stack Architecture:** Built using Spring Boot-based full-stack architecture following the MVC (Model-View-Controller) design pattern.

- **Frontend Layer:** Developed using Thymeleaf, HTML, CSS, and Bootstrap to create dynamic and responsive user interfaces.
- **Backend Layer:** Implemented using Java and Spring Boot to handle authentication, event management, and registration processing.
- **Database Layer:** MySQL is used as a relational database for storing users, events, and registration data.
- **Authentication System:** Secure login implemented using Spring Security with role-based access control (Admin and Student).
- **Server-Side Rendering:** Thymeleaf templates dynamically render data from backend to frontend.
- **State Management:** Session-based management is used to handle user authentication and access control.
- **Security Architecture:** Includes login authentication, role validation, and restricted access to protected pages.
- **Deployment Scope:** Designed for local deployment and can be accessed via browser using localhost.
- **Scalability Design:** Modular structure allows future enhancements such as cloud deployment and API integration.

1.6 Limitations and Out of Scope

- **No Payment Integration:** The system currently supports only free event registrations.
- **No Email Notifications:** No automated email or SMS alerts for event updates.

- **Limited Offline Support:** Requires internet or local server connection to access the system.
- **No Mobile Application:** The system is web-based and does not include Android or iOS apps.
- **Basic Analytics:** Only simple dashboards are available without advanced data analysis.
- **No Cloud Deployment:** The system is configured for local execution and not deployed on cloud platforms.
- **Limited Multi-User Roles:** Only Admin and Student roles are supported.
- **No Third-Party Integration:** Does not integrate with external platforms or services.
- **No Real-Time Notifications:** Updates are not pushed instantly to users.
- **Manual Attendance Tracking:** No automated attendance system like QR or biometric integration.

1.7 Frameworks and Technologies

CampusEvents is developed using modern web technologies with a Spring Boot backend and Thymeleaf-based frontend. The system follows a server-side rendering approach with secure authentication and efficient database handling using MySQL.

1.7.1 Backend Framework: Spring Boot

- **Spring Boot:** Main framework used for building the backend application.
- **Spring Security:** Provides authentication and role-based access control.
- **Spring Data JPA:** Handles database operations and ORM mapping.

- **Hibernate:** Maps Java objects to database tables.
- **Java:** Core programming language used for application logic.

1.7.2 Frontend Framework: Thymeleaf

- **Thymeleaf:** Server-side template engine used for dynamic page rendering.
- **HTML5:** Used for structuring web pages.
- **CSS3:** Used for styling and layout design.
- **Bootstrap:** Used for responsive design and UI components.

1.7.3 Frontend Styling

The frontend design is implemented using CSS3 and Bootstrap to provide a clean, responsive, and user-friendly interface. It ensures compatibility across different devices such as desktops, tablets, and mobile screens. Layouts are designed using grid systems and flexible components to enhance usability and visual appeal.

1.7.4 Database and Persistence: MySQL

- **MySQL:** Relational database used to store application data.
- **Tables Structure:** Includes Users, Events, and Registrations tables.
- **Data Persistence:** Stores user details, event information, and registrations securely.
- **Data Integrity:** Ensures consistency through structured relationships and constraints.
- **Scalability:** Can be extended for larger datasets and future enhancements.

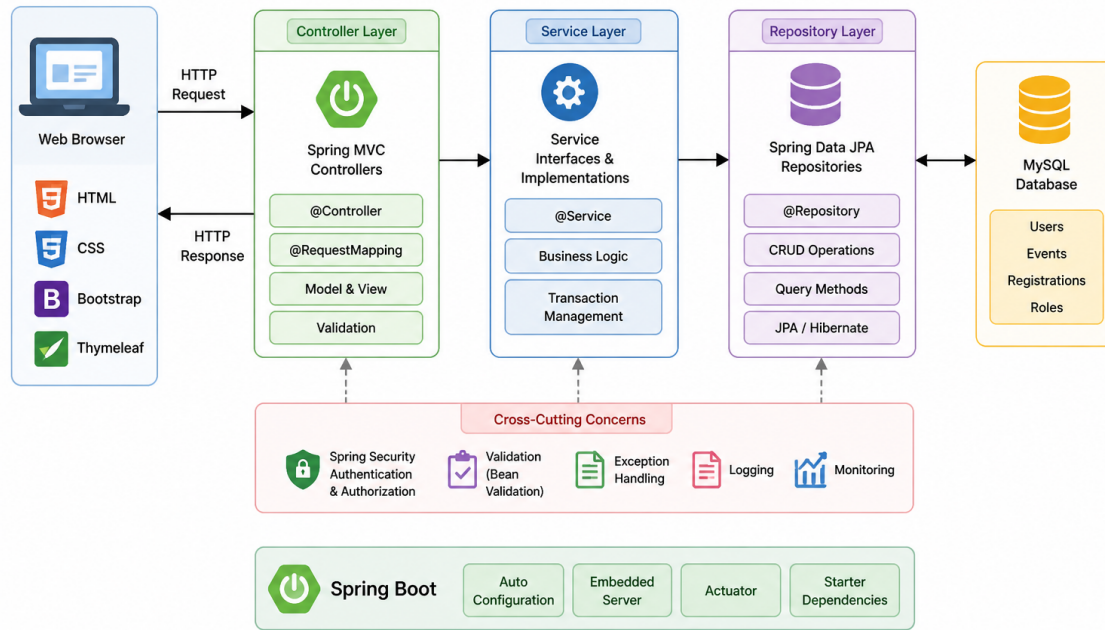


Figure 1.1: Architecture of CampusEvents System

The figure 1.1 shows the architecture of the CampusEvents system, which is designed as a role-based campus event management platform. The system follows a structured multi-layered architecture where users such as Admins and Students interact through a web-based interface. The presentation layer is developed using HTML, CSS, Bootstrap, and Thymeleaf, enabling dynamic server-side rendering and responsive design. This layer allows students to browse events and register, while administrators can create, update, and manage event details efficiently.

The backend is implemented using Spring Boot, which handles business logic, authentication, and data processing. Spring Security ensures secure role-based access control, separating admin and student functionalities. The data layer uses MySQL to store user information, event details, and registration records. All components are integrated in a monolithic architecture, ensuring smooth communication between layers, improved data consistency, and efficient system performance.

Chapter 2

Survey of Similar Applications in Market

The campus event management domain includes various platforms such as Eventbrite [1], Meetup [2], Google Forms [3], and internal university portals. These systems provide essential functionalities like event creation, registration handling, participant tracking, and notifications. They are widely used due to their ease of access and ability to manage large numbers of users. However, these platforms differ in terms of customization, security, cost, and suitability for campus-specific requirements.

While platforms like Eventbrite [1] and Meetup [2] focus on large-scale public event management and ticketing, tools like Google Forms [3] and traditional notice boards are commonly used within universities for simple event registration. Despite their usefulness, many existing systems lack proper role-based access control, real-time capacity tracking, and centralized dashboards tailored for academic environments. Compared to these systems, **CampusEvents** provides a secure, efficient, and customizable solution specifically designed for campus event management with improved control and user experience.

2.0.1 Large-Scale Public Event Platforms (Eventbrite, Meetup)

Major platforms such as Eventbrite and Meetup are widely used for organizing and managing public events across various domains.

- **Strengths:** Large user base, scalable infrastructure, advanced event discovery, and reliable ticketing systems.
- **Weaknesses:** Limited customization for campus use, lack of strict role-based access control, and dependency on external platforms.

2.0.2 University Traditional Methods (Google Forms, Notice Boards)

Many educational institutions rely on simple tools such as Google Forms and physical notice boards for managing event registrations.

- **Strengths:** Easy to use, no development cost, and quick deployment.
- **Weaknesses:** No centralized management, lack of real-time updates, risk of duplicate entries, and poor data organization.

2.0.3 Modern Web-Based Systems

Modern web-based applications developed using technologies such as Java, Spring Boot, and MySQL offer improved performance, security, and flexibility for managing event-related data.

- **Strengths:** Dynamic user interfaces, secure authentication, modular design, and scalability.
- **Weaknesses:** Some systems lack real-time tracking features, advanced analytics, and personalized user dashboards.

2.0.4 Gap Analysis and CampusEvents Advantage

A detailed comparison of existing systems shows that most platforms either focus on large-scale public events or basic data collection, but fail to combine security, real-time tracking, and centralized management in a single system. **CampusEvents** bridges this gap by integrating role-based authentication, real-time capacity tracking, and a centralized dashboard for both administrators and students. It provides better control, improved data accuracy, and enhanced user experience while being cost-effective and scalable. float

Feature	Eventbrite	Google Forms	Meetup	CampusEvents
Role-Based Access	No	Limited	No	Yes
Event Registration	Yes	Yes	Yes	Yes
Centralized Dashboard	Yes	No	Yes	Yes
Real-Time Capacity Tracking	Limited	No	Limited	Yes
Customization	Low	Medium	Low	High
Data Security	Medium	Low	Medium	High
Scalability	High	Medium	High	High
Automation	Medium	Low	Medium	High

Table 2.1: Comparative Analysis of Event Management Systems

Table 2.1 represents a comparative analysis of various event management systems, highlighting key features such as role-based access, event registration, and system capabilities. The comparison shows that existing platforms lack customization and strong access control.

Chapter 3

Framework Description

3.1 Frontend Implementation

The frontend of CampusEvents is developed using HTML, CSS, Bootstrap, and Thymeleaf to provide a responsive and interactive user interface. It enables users to browse events, view event details, and register seamlessly. The interface is designed to ensure ease of use, fast rendering, and compatibility across multiple devices.

3.1.1 Environment Setup

- **HTML5:** Provides the structural foundation for web pages.
- **CSS3:** Used for styling, layout design, and responsive behavior.
- **Bootstrap:** Helps in building a responsive and modern UI design.
- **Thymeleaf:** Used for dynamic server-side rendering of web pages.

3.1.2 Frontend Features

- **Event Display:** Dynamically shows available events with details such as date, location, and capacity.
- **Student Dashboard:** Allows students to view and manage registered events.
- **Admin Dashboard:** Provides centralized control for managing events and tracking registrations.
- **Responsive UI:** Ensures proper display across desktops, tablets, and mobile devices.
- **User-Friendly Navigation:** Simplifies browsing and accessing different modules.

3.2 Backend Implementation

The backend of CampusEvents is developed using Java and Spring Boot, following the MVC architecture. It manages business logic, processes user requests, and ensures smooth communication between frontend and database.

3.2.1 Environment Setup

- **Spring Boot:** Core framework used to build the backend application.
- **Spring Security:** Handles authentication and role-based access control.
- **Spring Data JPA:** Simplifies database interaction and ORM mapping.

3.2.2 Backend Features

- **User Authentication:** Secure login system for Admin and Student users.
- **Event Management:** Allows creation, updating, and deletion of events.
- **Registration Handling:** Manages student registrations and prevents overbooking.
- **Business Logic Processing:** Handles validation and application workflows.
- **Error Handling:** Ensures proper validation and smooth execution of operations.

3.3 Database Implementation

The database of CampusEvents is implemented using MySQL, providing a structured and reliable system for storing and managing data.

3.3.1 Database Configuration

- **MySQL:** Stores structured data using relational tables.

- **Tables:** Includes Users, Events, and Registrations.
- **Data Storage:** Maintains records of users, event details, and registrations.

3.3.2 Database Schema Structure

- **Users Table:** Stores user credentials and roles.
- **Events Table:** Contains event information such as title, date, and capacity.
- **Registrations Table:** Tracks student participation in events.

float

3.4 System Technology Stack Overview

Component	Technology	Description
Frontend	HTML, CSS, Bootstrap, Thymeleaf	Builds dynamic and responsive UI
Styling	CSS3, Bootstrap	Provides layout and visual design
Rendering	Thymeleaf	Server-side template rendering
Backend	Spring Boot (Java)	Handles application logic
Database	MySQL	Stores structured application data
Authentication	Spring Security	Secure login and role-based access

Table 3.1: System Technology Stack for CampusEvents

Table 3.1 represents the technology stack used in the CampusEvents system, outlining the key components and their roles in the application. The frontend is developed using HTML, CSS, Bootstrap, and Thymeleaf to provide a responsive and dynamic user interface, while Thymeleaf enables server-side rendering for better performance and security.

The backend is implemented using Spring Boot, which manages application logic and integrates with MySQL for efficient data storage.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The architecture of **CampusEvents** is designed using a **Multi-Tier Web Architecture** that ensures scalability, modularity, and efficient system performance. The system separates responsibilities such as user interface, business logic, security, and database operations into independent layers. This separation improves maintainability and allows future enhancements without affecting the entire system [4][7].

The platform follows a client-server model in which the frontend, built using HTML [11], CSS [12], Bootstrap [10], and Thymeleaf [6], interacts with the backend developed in Spring Boot [4]. This approach enables smooth data exchange and reduces system dependency between components. It also supports server-side rendering, ensuring fast and secure page delivery without heavy client-side processing [6][4].

- **Presentation Layer (Frontend):** This layer is responsible for user interaction and visual representation. It is developed using HTML5, CSS3, Bootstrap, and Thymeleaf to provide a responsive and user-friendly interface. It includes login pages, dashboards, event listings, and registration interfaces.
- **Application Layer (Backend):** Implemented using Spring Boot, this layer handles core business logic such as authentication, event management, and registration processing. It processes user requests and communicates with the database[4].
- **Security Layer:** Ensures secure access using Spring Security with BCrypt

password encryption. It enforces role-based access control (Admin and Student) and protects sensitive data from unauthorized access[5].

- **Data Layer (Database):** MySQL database is used to store structured data including user details, event information, and registration records. Proper relational mapping ensures efficient data retrieval and consistency[8].
- **Server-Side Rendering Layer:** Thymeleaf templates dynamically generate HTML pages by binding backend data before sending them to the client, ensuring faster and secure rendering[6].

The architecture is designed to be flexible and scalable, allowing integration of advanced features such as cloud deployment, analytics dashboards, and notification systems in the future.

4.2 Data Flow Diagram (DFD)

4.2.1 DFD Level 0: Context Diagram

The Level 0 Data Flow Diagram provides a high-level overview of the entire system. It represents CampusEvents as a single process interacting with external entities such as Students and Administrators.

Students interact with the system by registering, logging in, browsing events, and registering for events. Administrators manage event creation, updates, and monitor registrations.

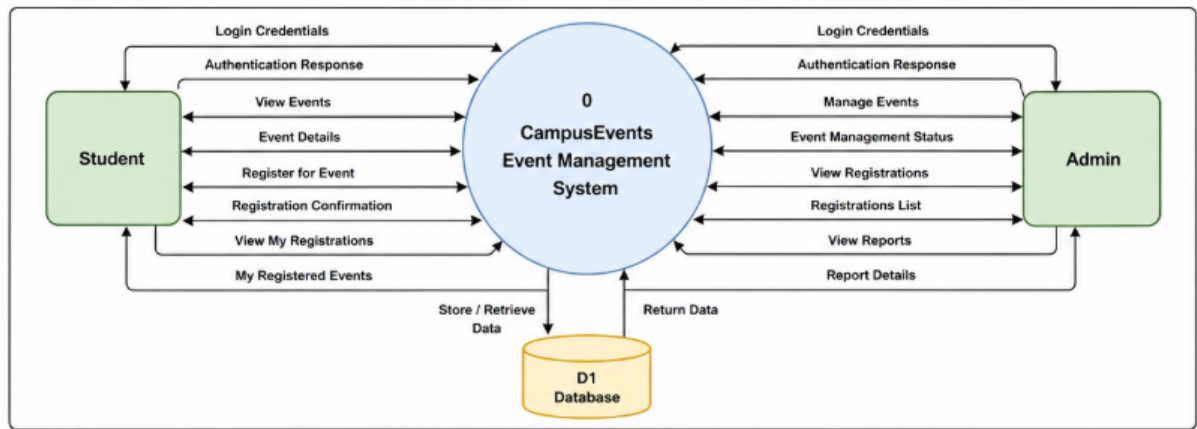


Figure 4.1: Data Flow Diagram Level 0

The figure 4.1 shows the Level 0 Data Flow Diagram of the CampusEvents system, which provides a high-level overview of the entire system. It represents the system as a single process interacting with external entities such as Students and Administrators.

4.2.2 DFD Level 1: Functional Decomposition

The Level 1 DFD breaks down the system into multiple internal processes such as Authentication Module, Event Management Module, Event Browsing Module, Registration Module, and Dashboard System.

Each module performs specific tasks and interacts with the MySQL database to store and retrieve data. This decomposition helps in understanding the flow of data within the system and improves system organization.

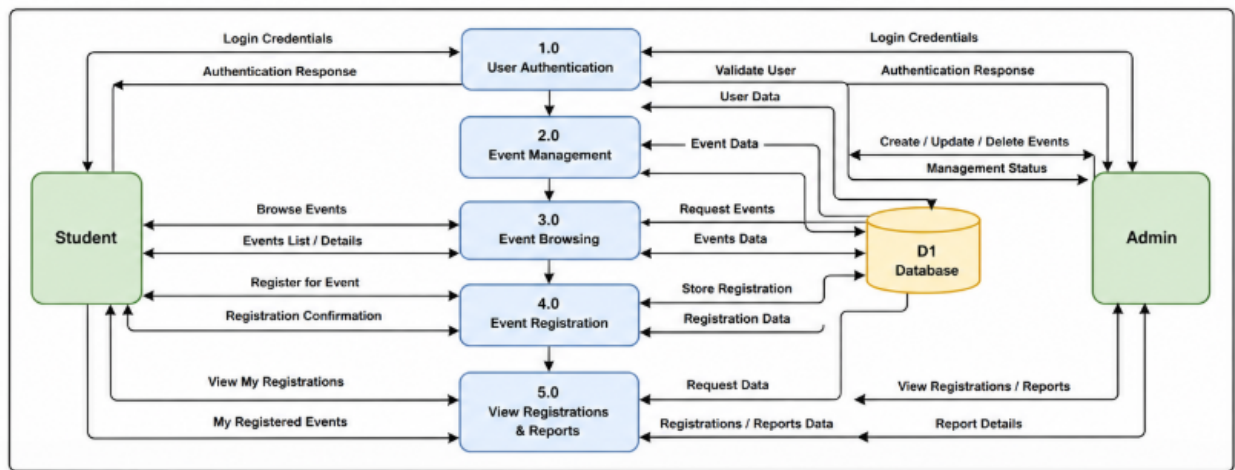


Figure 4.2: Data Flow Diagram Level 1

The figure 4.2 shows the Level 1 Data Flow Diagram of the CampusEvents system, which provides a detailed view of the internal processes involved in the system. It breaks down the main system into multiple functional modules such as Authentication, Event Management, and Registration Processing.

The Authentication module handles user login and verification for both students and administrators. The Event Management module allows administrators to create, update, and delete events, while the Registration module enables students to register for events and stores their participation details in the database.

The diagram illustrates how data flows between these modules and the database, ensuring proper communication and processing within the system. This detailed representation helps in understanding how individual components work together to deliver the complete functionality of the CampusEvents platform.

4.3 System Modules and Functional Architecture

The **CampusEvents** system is divided into multiple modules, each responsible for a specific functionality. This modular approach ensures better organization, easier debugging, and scalability.

4.3.1 Module 1: User Authentication and Security

This module ensures secure user access and identity verification.

Functionality:

- User login system for Admin and Student roles.
- Password encryption using BCrypt hashing.
- Session management for maintaining user state.
- Role-based access control.

Benefits:

- Enhances system security and user data protection.
- Prevents unauthorized access.
- Ensures reliable authentication across modules.

4.3.2 Module 2: Event Management Module

This module allows administrators to manage event details efficiently.

Functionality:

- Create new events.
- Update event details such as date, location, and description.
- Delete events when required.
- Maintain organized event listings.

4.3.3 Module 3: Event Browsing Module

This module allows students to explore available events.

Functionality:

- View list of upcoming events.

- Access event details.
- Navigate events easily.

4.3.4 Module 4: Registration Module

This module manages student participation in events.

Functionality:

- Register for events.
- Store registration details.
- Prevent duplicate registrations.
- Track participation records.

4.3.5 Module 5: Dashboard Module

This module provides personalized system views.

Functionality:

- Student dashboard for viewing registered events.
- Admin dashboard for monitoring events and registrations.
- Display system summaries.

4.3.6 Module 6: Capacity Tracking Module

This module ensures proper event participation limits.

Functionality:

- Track number of registrations per event.
- Prevent overbooking.
- Maintain accurate event capacity.

4.3.7 Module 7: Database Management Module

This module handles data storage and retrieval.

Functionality:

- Store users, events, and registrations.
- Maintain relational integrity.
- Provide efficient data access.

4.4 Advantages and Limitations of the Project

4.4.1 Advantages

- **Centralized Event Management:** All event data is maintained in a single platform.
- **Automation:** Eliminates manual registration processes.
- **Secure System:** Ensures safe authentication and role-based access.
- **User-Friendly Interface:** Easy navigation and responsive design.
- **Scalable Architecture:** Supports future enhancements and system expansion.
- **Real-Time Tracking:** Provides instant updates on event registrations.
- **Efficient Data Management:** Ensures accuracy and consistency of data.

4.4.2 Limitations

- **Internet Dependency:** Requires active network connection.
- **No Payment Integration:** Does not support paid event registrations.
- **Basic Notification System:** No automated email or SMS alerts.
- **Monolithic Architecture Constraints:** Limited scalability compared to microservices.

Chapter 5

RESULTS AND DISCUSSIONS

5.1 System Implementation Overview

The **CampusEvents** system was successfully implemented as a dynamic web-based application that integrates a responsive frontend with a Spring Boot-based backend [4] and MySQL database [8]. The system is designed to manage campus events, registrations, and user roles efficiently using secure authentication [5] and real-time updates. The implementation demonstrates the use of modern web technologies to create a scalable and user-friendly event management platform. It enables both students and administrators to interact with the system through dedicated interfaces while ensuring smooth data flow and secure operations.

The major modules implemented in the system include:

- Landing Page Interface
- User Authentication Module (Admin/Student Login)
- Event Browsing System
- Event Registration System
- Student Dashboard
- Admin Dashboard
- Event Management Module
- Capacity Tracking System
- Database Layer

The frontend of the application is developed using HTML5 [11], CSS3 [12], Bootstrap [10], and Thymeleaf [6] to provide a responsive and interactive user experience.

The backend is implemented using Spring Boot [4], which handles authentication, event operations, and registration processing. MySQL [8] is used for structured data storage, ensuring efficient retrieval and consistency.

Secure authentication is implemented using Spring Security [5] with role-based access control. The admin module provides full control over event creation and monitoring, while students can browse events and register efficiently.

5.2 System Interface Visualizations

The system interface of **CampusEvents** is designed to provide a modern, responsive, and user-friendly experience for both students and administrators. The interfaces are visually structured with a clean layout, intuitive navigation, and dynamic data representation to enhance usability and system interaction [10][6].

The platform includes multiple interconnected interfaces such as the login page, student dashboard, event browsing interface, registration page, and admin dashboard. Each interface is designed to perform specific functions while maintaining consistency in design and performance.

The login interface serves as the entry point of the application, where users authenticate using secure credentials. It ensures safe access and prevents unauthorized usage using Spring Security mechanisms [5].

The screenshot displays the 'Raw Event Management Login' interface. At the top left, the site logo 'Raw Event Management' is visible. At the top right, the user is logged in as 'admin@rawevents.com' with an 'ADMIN' role indicator. The login form itself is centered and contains a red header with the title 'Raw Event Management Login'. Below the header, a pink message box states 'You have been logged out.' The form includes an 'Email address' field with 'admin@rawevents.com' entered, a 'Password' field with masked characters '*****', and a 'Remember me' checkbox which is currently unchecked. A red 'Login' button is positioned below the password field. At the bottom of the form, there is a link that reads 'Don't have an account? [Register here](#)'.

Figure 5.1: Admin Authentication

The figure 5.1 describes the admin authentication interface of the CampusEvents system, which ensures secure access for administrators. The login page requires valid credentials and verifies user identity using Spring Security, preventing unauthorized access to administrative functionalities.

This interface acts as the entry point for system control, allowing only authorized users to manage events, view registrations, and perform administrative operations securely.

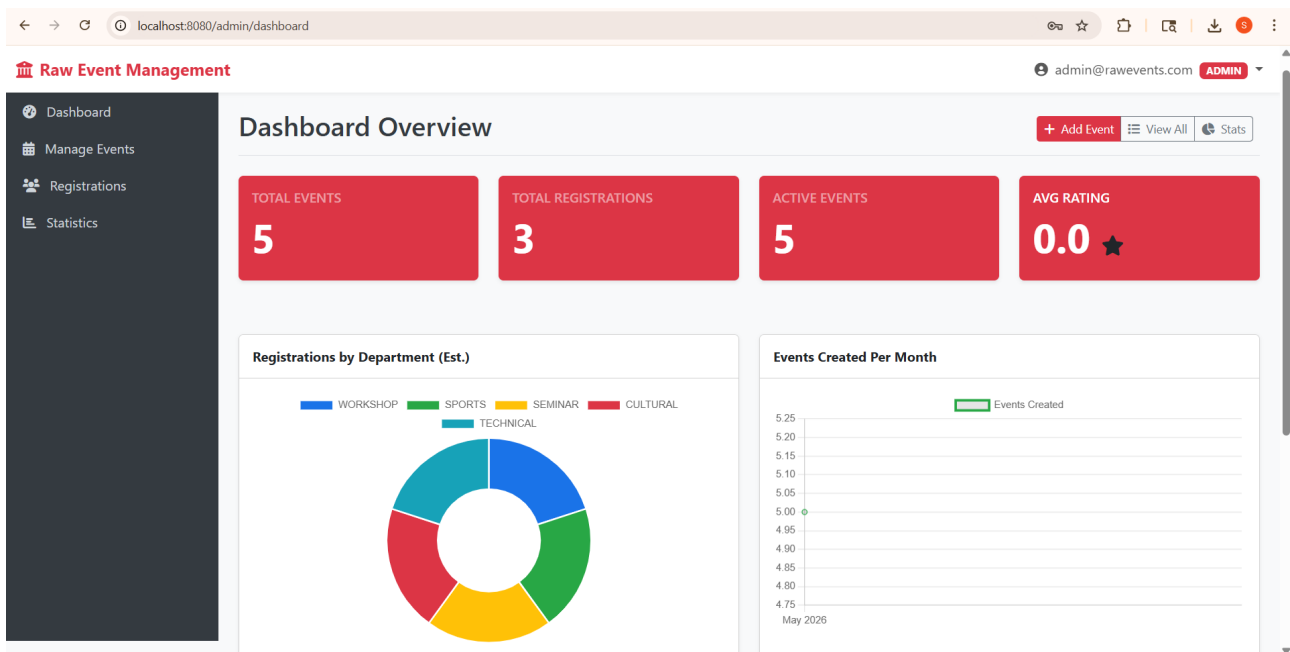


Figure 5.2: Admin Dashboard

The figure 5.2 describes the admin dashboard of the CampusEvents system, which acts as the central interface for managing all event-related activities. It provides administrators with tools to create, update, and delete events, ensuring efficient control over the system.

The dashboard displays important information such as event details, number of registered students, and overall system status. This real-time data helps administrators monitor participation and make effective decisions regarding event planning and management.

Additionally, the interface is designed to be user-friendly and responsive, allowing smooth navigation between different functionalities. The admin dashboard enhances productivity by centralizing all operations in a single platform, making event management more efficient and organized.

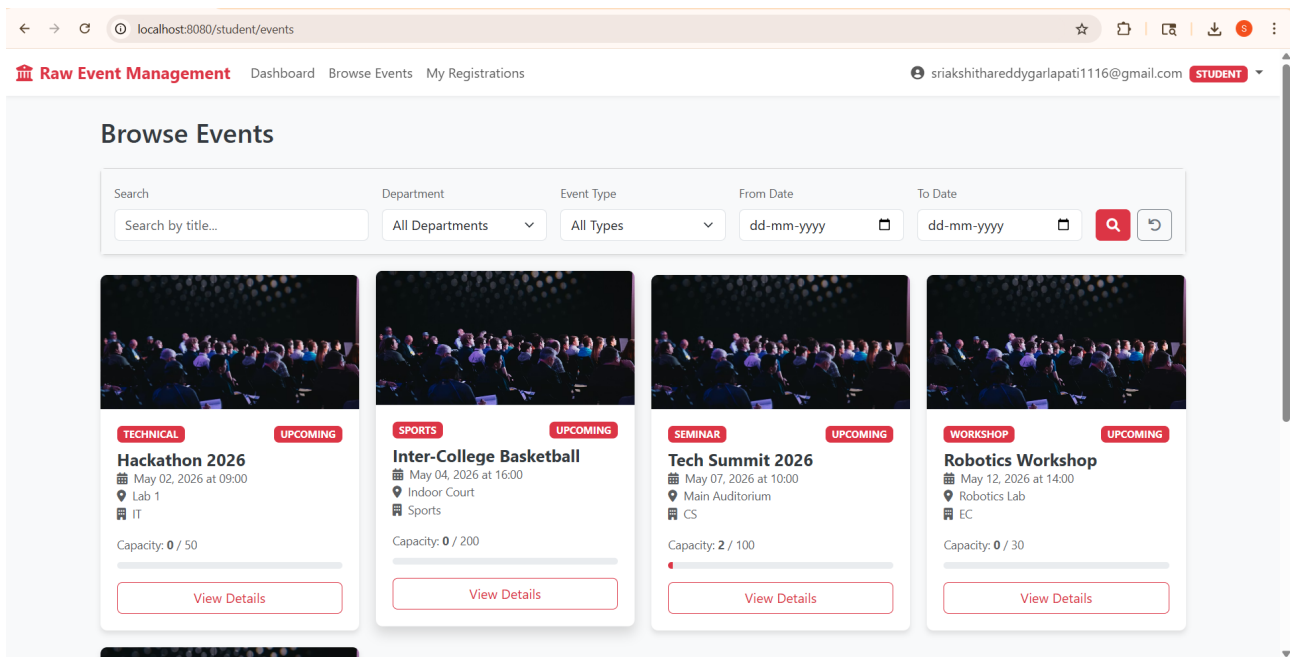


Figure 5.3: Student Event Browsing

The figure 5.3 describes the student event browsing interface of the CampusEvents system, which allows users to explore available events in a structured and user-friendly manner. The interface displays essential event details such as title, date, location, and brief descriptions, helping students easily understand each event.

This module is designed to provide smooth navigation and quick access to upcoming activities. Students can browse through multiple events and select those that match their interests. The responsive design ensures accessibility across different devices, including desktops and mobile platforms.

By presenting event information clearly and efficiently, this interface enhances user experience and encourages student participation. It acts as a key component in the system by bridging the gap between event creation and student engagement.

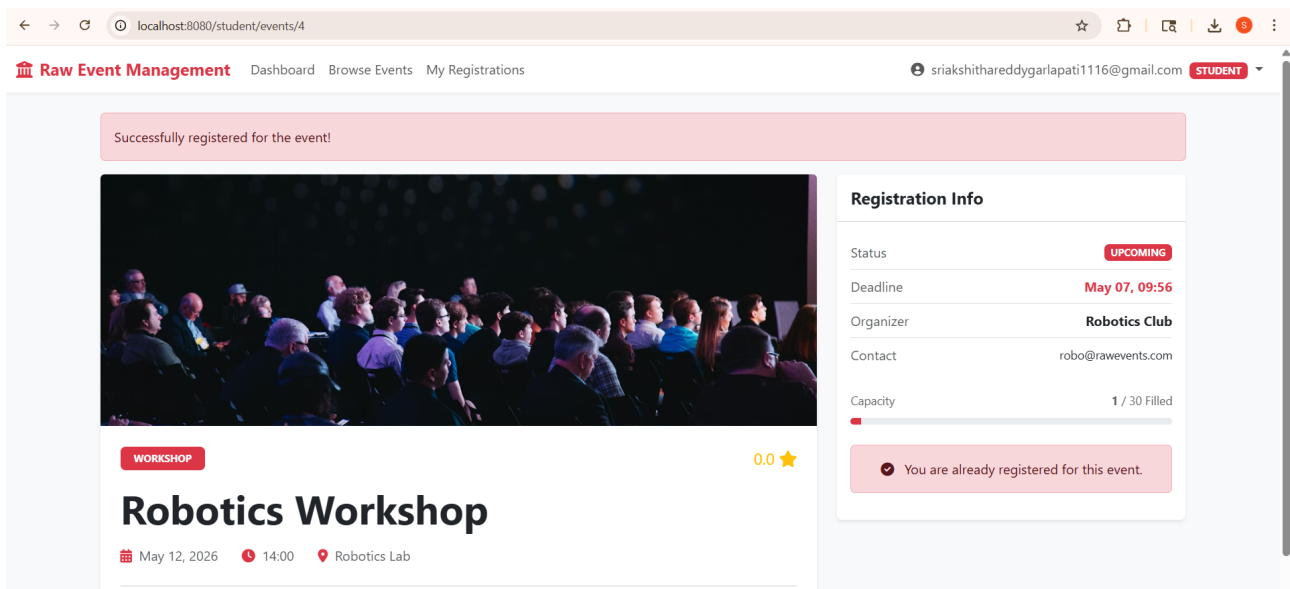


Figure 5.4: Event Registration Page

The figure 5.4 describes the event registration page of the CampusEvents system, where students can register for selected events through a simple and user-friendly interface. The page displays relevant event details and provides an option for users to confirm their participation.

The registration module securely captures user data and maps students to the selected events using backend processing. It ensures that each student can register only once for an event, preventing duplicate entries and maintaining data integrity within the system.

Additionally, the system tracks the number of registrations for each event and enforces capacity limits to avoid overbooking. This module plays a crucial role in automating the registration process and improving overall efficiency in event management.

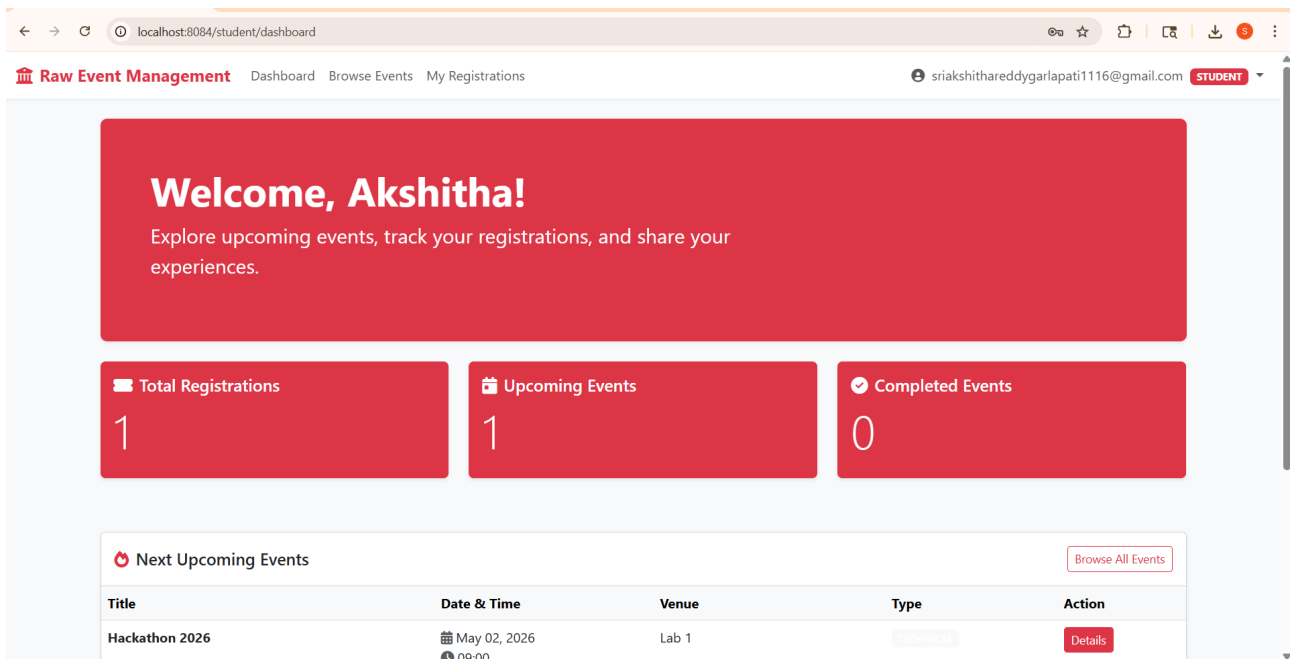


Figure 5.5: Student Dashboard

The figure 5.5 describes the student dashboard of the CampusEvents system, which provides a personalized interface for users after successful login. It allows students to view their profile information and access all event-related functionalities in one place.

The dashboard displays the list of events that the student has registered for, along with relevant details such as event name, date, and status. This helps users keep track of their participation and stay informed about upcoming activities.

The interface is designed to be simple, responsive, and easy to navigate, ensuring a smooth user experience across devices. By centralizing all student-related information, the dashboard enhances usability and encourages active participation in campus events.

5.3 Module Explanations

The **CampusEvents** system is designed using a modular approach where each module performs a specific function, improving scalability and maintainability.

5.3.1 Landing Page Interface

The landing page serves as the entry point of the system. It provides navigation options and quick access to login and event browsing features.

5.3.2 User Authentication Module

This module ensures secure login using Spring Security. It supports role-based access control for Admin and Student users and maintains session security.

5.3.3 Event Browsing Module

This module allows students to view available events dynamically with details such as title, date, and location.

5.3.4 Event Details Module

Displays complete information about selected events, helping students understand event details before registration.

5.3.5 Event Registration Module

This module allows students to register for events. It ensures proper mapping between students and events and prevents duplicate registrations.

5.3.6 Student Dashboard Module

Provides a personalized interface where students can view registered events and participation details.

5.3.7 Admin Dashboard Module

Acts as the central control panel where administrators can monitor events and registrations.

5.3.8 Event Management Module

Allows administrators to create, update, and delete events efficiently.

5.3.9 Capacity Tracking Module

Ensures that event registrations do not exceed the maximum limit, preventing overbooking.

5.3.10 Database Management Module

Manages data storage using MySQL for users, events, and registrations.

5.3.11 Search and Filtering Module

Allows students to easily find events using simple browsing and navigation features.

5.3.12 Security Module

Ensures system protection using authentication and role-based access control.

5.3.13 System Logging Module

Tracks system activity for monitoring and improving system performance.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The **CampusEvents: Role-Based Campus Event Management System** successfully demonstrates the development of a secure, efficient, and user-friendly web-based application using modern full-stack technologies. The project integrates a robust **Spring Boot backend** [4] with dynamic **Thymeleaf-based frontend** [6] to deliver a seamless experience for both administrators and students.

The system effectively implements role-based access control using **Spring Security** [5], ensuring secure authentication and proper separation between admin and student functionalities. The admin dashboard enables efficient management of events, including creation, updating, deletion, and monitoring of registrations, while the student interface provides features such as event browsing, registration, and participation tracking.

One of the key highlights of the system is **real-time capacity tracking**, which prevents overbooking and ensures fair allocation of event slots. The centralized platform eliminates manual processes and reduces errors associated with traditional event management systems [3]. The application also ensures smooth communication between frontend and backend through server-side rendering [6], enabling efficient data handling and quick response times.

The implementation demonstrates that a well-structured MVC architecture improves maintainability, scalability, and overall system performance [4]. Overall, the CampusEvents system serves as a reliable and effective solution for digitizing

campus event management, improving transparency, and enhancing student engagement.

6.2 Future Enhancements

Although the current implementation provides essential event management functionalities, several advanced features can be incorporated in future versions to improve scalability, performance, and user experience.

- **Online Payment Integration:** Adding secure payment gateway support (e.g., Razorpay or Stripe) for paid events and ticketing.
- **Cloud-Based Deployment:** Deploying the application on cloud platforms such as AWS or Azure to enhance scalability, availability, and accessibility.
- **Email Notifications and Alerts:** Implementing automated email notifications for event registration confirmations, reminders, and updates.
- **QR Code-Based Attendance System:** Introducing QR code scanning for efficient and contactless attendance tracking during events.
- **Advanced Analytics Dashboard:** Providing graphical insights such as event participation trends, user activity, and event popularity.
- **Mobile Application Development:** Developing a mobile application using Flutter or React Native to improve accessibility and user engagement.
- **Multi-Role Support:** Extending the system to include additional roles such as faculty coordinators or event organizers.
- **Smart Recommendation System:** Implementing AI-based suggestions to recommend events to students based on their interests and past participation.

6.3 Source Code

This section presents key components of the CampusEvents system including controller, service, entity, repository, and frontend template.

Listing 6.1: Event Controller

```
@Controller
@RequestMapping("/events")
public class EventController {

    @Autowired
    private EventService eventService;

    @GetMapping
    public String getAllEvents(Model model) {
        model.addAttribute("events", eventService.getAllEvents());
        return "events";
    }

    @GetMapping("/new")
    public String showCreateForm(Model model) {
        model.addAttribute("event", new Event());
        return "create_event";
    }

    @PostMapping("/save")
    public String saveEvent(@ModelAttribute Event event) {
        eventService.saveEvent(event);
        return "redirect:/events";
    }

    @GetMapping("/delete/{id}")
    public String deleteEvent(@PathVariable Long id) {
        eventService.deleteEvent(id);
        return "redirect:/events";
    }
}
```

Listing 6.2: Event Service

```
@Service
public class EventService {

    @Autowired
    private EventRepository eventRepository;
```

```

    public List<Event> getAllEvents () {
        return eventRepository.findAll ();
    }

    public void saveEvent(Event event) {
        eventRepository.save(event);
    }

    public void deleteEvent(Long id) {
        eventRepository.deleteById(id);
    }
}

```

Listing 6.3: Event Entity

```

@Entity
public class Event {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String description;
    private String date;
    private int capacity;

    // Getters and Setters
}

```

Listing 6.4: Event Repository

```

@Repository
public interface EventRepository extends JpaRepository<Event, Long> {
}

```

Listing 6.5: Event Listing Page (Thymeleaf)

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Events</title>
</head>
<body>

<h2>Available Events</h2>

<table>
    <tr>

```

```

    <th>Title</th>
    <th>Date</th>
    <th>Capacity</th>
</tr>

<tr th:each="event:#{events}">
    <td th:text="{event.title}"></td>
    <td th:text="{event.date}"></td>
    <td th:text="{event.capacity}"></td>
</tr>

</table>

</body>
</html>

```

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

The **CampusEvents: Role-Based Campus Event Management System** addresses multiple complex engineering problems by integrating secure authentication, scalable architecture, responsive user interfaces, and efficient data management into a unified digital platform.

The project combines frontend technologies (Thymeleaf, HTML, CSS, Bootstrap), backend services (Spring Boot), authentication mechanisms (Spring Security), and database operations (MySQL) while maintaining system scalability, reliability, and performance.

Level	Attribute	Characteristics	Wks	Justification
WP1	Depth of Knowledge	Requires strong knowledge in full-stack development and system design	WK3, WK4, WK5, WK6	The project involves Spring Boot, Thymeleaf, MVC architecture, Spring Security, and MySQL database design.
WP2	Conflicting Requirements	Balancing usability, security, and performance	WK4, WK5, WK6	The system ensures a user-friendly interface while maintaining secure authentication and efficient backend processing.

WP3	Depth of Analysis	System-level design and logical processing	WK3, WK4, WK5	Includes database schema design, event handling logic, and role-based access control.
WP4	Familiarity	Moderately complex engineering challenges	WK4, WK8	Implementing secure authentication and real-time capacity tracking introduces complexity beyond basic applications.
WP5	Applicable Codes	Custom logic implementation	WK5, WK6	Custom logic is implemented for event creation, registration management, and user authentication workflows.
WP6	Stakeholder Needs	Multiple user roles and requirements	WK5, WK7	The system supports administrators and students with different functionalities and access levels.
WP7	Interdependence	Integration of multiple subsystems	WK3, WK5, WK6	The system integrates frontend views, backend logic, security mechanisms, and database operations into a unified architecture.

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

Table 7.1 illustrates the mapping of the CampusEvents project to Complex Engineering Problems (CEP), highlighting key attributes such as depth of knowledge and conflicting requirements involved in system design and implementation.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

The development of the **CampusEvents** platform involves several complex engineering activities related to full-stack web development, secure system design, and database integration.

The project integrates frontend rendering, backend processing, authentication systems, and relational data management into a cohesive system capable of handling real-time event operations.

Additionally, the system applies modern engineering practices such as Role-Based Access Control (RBAC), secure authentication using Spring Security, and modular MVC architecture.

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of multiple modern technologies	The project uses Spring Boot, Thymeleaf, HTML, CSS, Bootstrap, MySQL, and Spring Security.
EA2	Level of Interactions	Complex module interaction	The system manages interaction between authentication, event management, registration processing, and database modules.
EA3	Innovation	Modern event management system	Provides role-based dashboards, real-time capacity tracking, and centralized management.
EA4	Consequences to Society	Impact on campus management	Improves event organization efficiency and enhances student participation in campus activities.
EA5	Familiarity	Beyond basic applications	Combines frontend rendering, backend logic, authentication, and database integration into a scalable system.

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

Table 7.2 demonstrates additional CEP attributes of the CampusEvents system, emphasizing system-level analysis, stakeholder requirements, and integration of multiple subsystems.

7.3 SDG Mapping (CEP Section)

The **CampusEvents System** contributes to multiple United Nations Sustainable Development Goals (SDGs) by promoting digital transformation, efficient resource management, and improved campus engagement.

The platform provides a centralized digital solution for managing academic and extracurricular events, improving accessibility and transparency.

By integrating secure authentication, structured event workflows, and scalable technologies, the system supports sustainable digital infrastructure within educational institutions.

SDG No.	SDG Title	Relevance
SDG 4	Quality Education	Supports student engagement through structured participation in academic and extracurricular activities.
SDG 9	Industry, Innovation and Infrastructure	Encourages adoption of modern digital systems for efficient campus management.
SDG 11	Sustainable Cities and Communities	Promotes organized and efficient event management within campus environments.
SDG 12	Responsible Consumption and Production	Reduces paper usage by digitizing event registration and management processes.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure access through role-based authentication and controlled system operations.

Table 7.3: SDG Mapping for CampusEvents

Table 7.3 illustrates the mapping of the CampusEvents project to Complex Engineering Activities (CEA), showing how various engineering practices and system interactions contribute to the overall solution.

References

- [1] Eventbrite: Event Management and Ticketing Platform.

<https://www.eventbrite.com>

Reference Note: Provides large-scale event management, ticketing, and participant tracking features.

- [2] Meetup: Event and Community Platform.

<https://www.meetup.com>

Reference Note: Enables group-based event organization and community engagement.

- [3] Google Forms: Online Form and Registration Tool.

<https://forms.google.com>

Reference Note: Commonly used for simple event registrations with limited automation and tracking.

- [4] Spring Boot Official Documentation.

<https://spring.io/projects/spring-boot>

Reference Note: Used for developing the backend application and handling business logic.

- [5] Spring Security Documentation.

<https://spring.io/projects/spring-security>

Reference Note: Provides authentication and role-based access control for secure system operation.

- [6] Thymeleaf Documentation.

<https://www.thymeleaf.org>

Reference Note: Used for server-side rendering of dynamic web pages.

[7] Spring Data JPA Documentation.

<https://spring.io/projects/spring-data-jpa>

Reference Note: Facilitates database interaction and ORM mapping.

[8] MySQL Official Documentation.

<https://dev.mysql.com/doc/>

Reference Note: Relational database used for storing users, events, and registrations.

[9] Hibernate ORM Documentation.

<https://hibernate.org/orm/documentation/>

Reference Note: Provides object-relational mapping between Java classes and database tables.

[10] Bootstrap Documentation.

<https://getbootstrap.com>

Reference Note: Used for designing responsive and user-friendly interfaces.

[11] HTML5 Documentation (MDN Web Docs).

<https://developer.mozilla.org/en-US/docs/Web/HTML>

Reference Note: Provides the structure for web pages.

[12] CSS3 Documentation (MDN Web Docs).

<https://developer.mozilla.org/en-US/docs/Web/CSS>

Reference Note: Used for styling and layout of the application.